

C系列の言語仕様

C系列の言語仕様

C言語 → C++言語 → C#言語

C系列はC言語から順番に改良されてC#になっています。

C言語： 手続き型

プログラムに区分けがなく複数人での開発が大変

C++言語： +オブジェクト指向

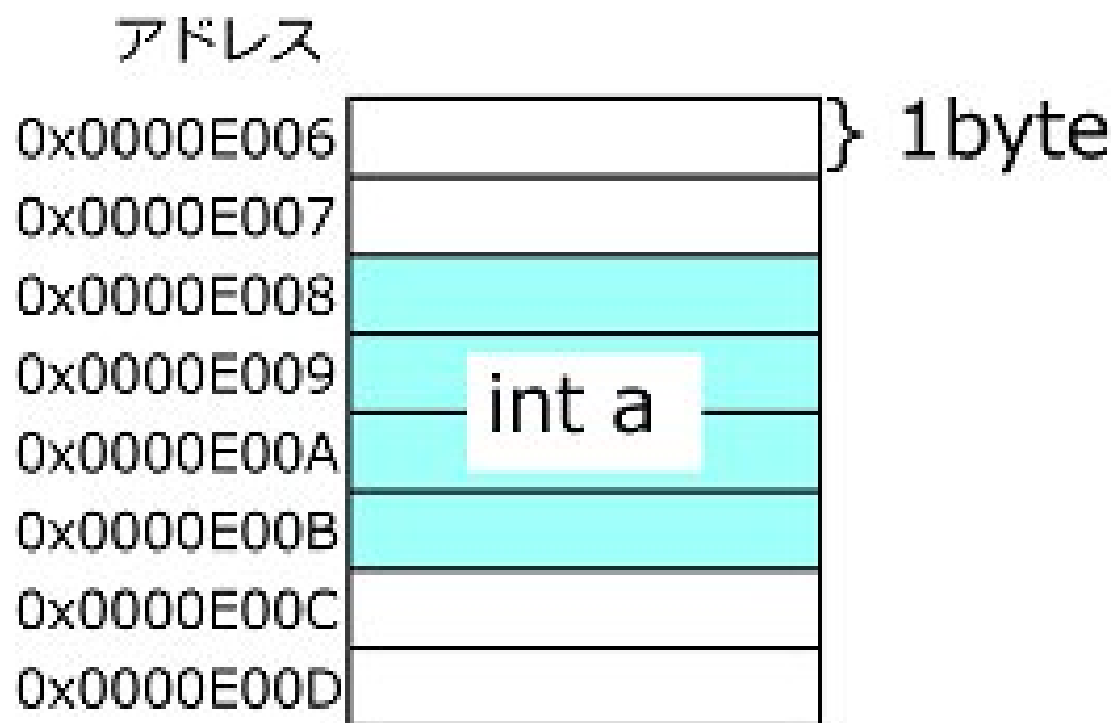
オブジェクト指向によるカプセル化で大規模開発がやりやすくなる
ポインターやメモリリークなど難しくなりすぎた。

C#言語： +イベント指向 +ガベージコレクション

ポインターなどをプログラム側で管理して人間の負担を軽減
C++に比べて処理が重たくなる。

メモリー

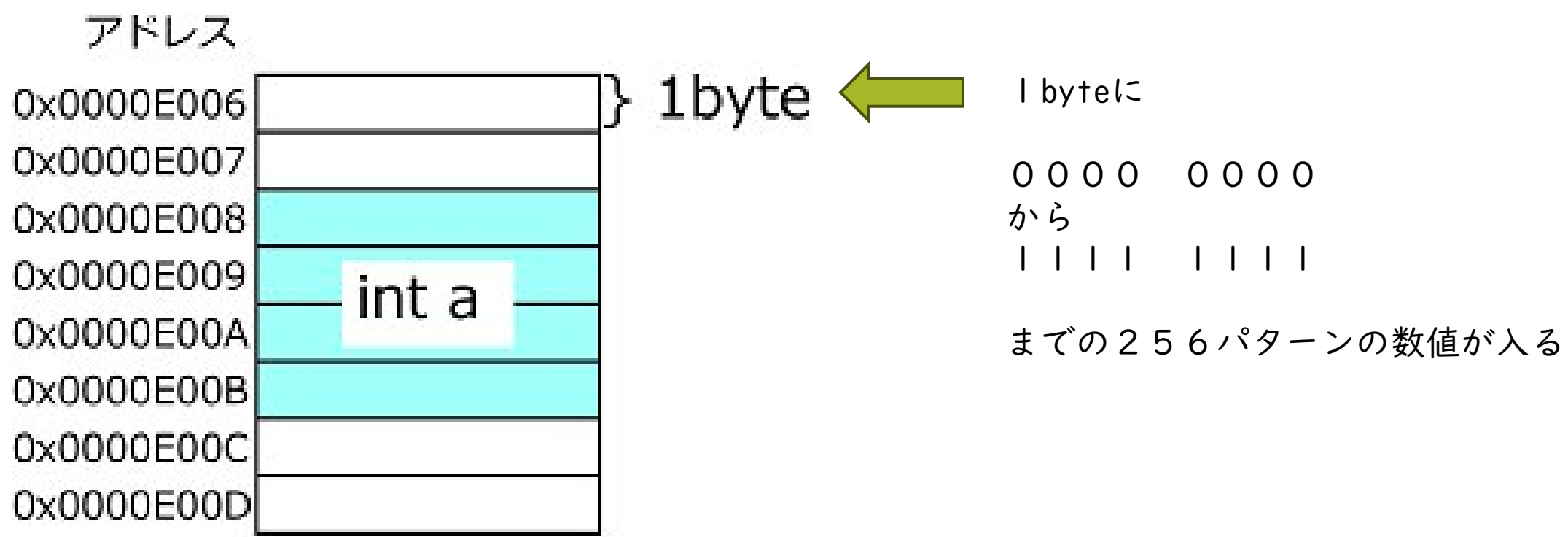
皆さんがプログラムで使用する変数はメモ리를消費しています。
メモリーにはポインターと言われるメモリの住所のような物と、
実際の値をいれる領域の2つで構成されています。



C系列の言語仕様

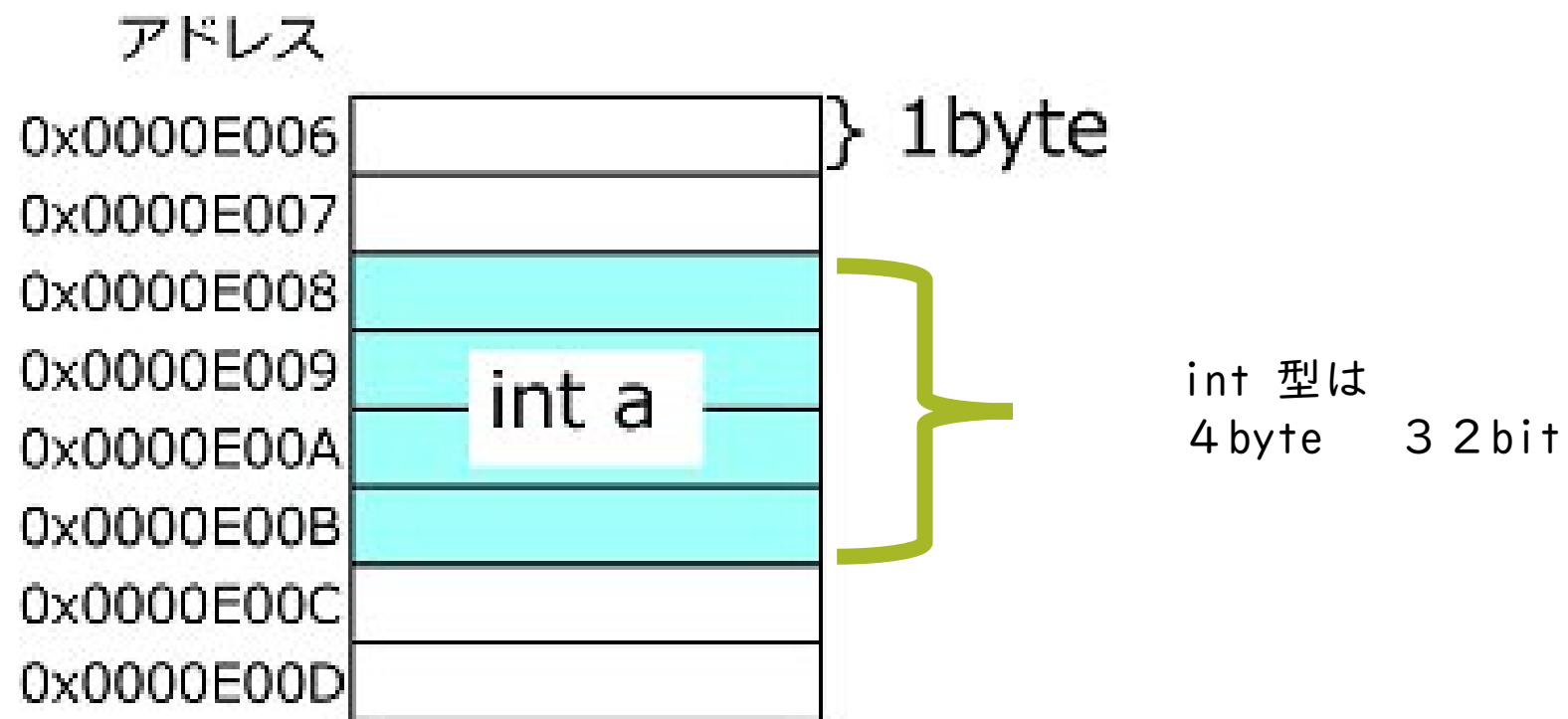
1 byte 1 bit

パソコンのデータは最小単位で見るとすべて0と1のみで構成されています。1 bitがその0 or 1で1 bitだと小さすぎるので最低限使いやすいサイズにしたのが、1 byteで 1 byte = 8 bit です。



変数

変数にはそれぞれメモリーを使う量が決まっています。

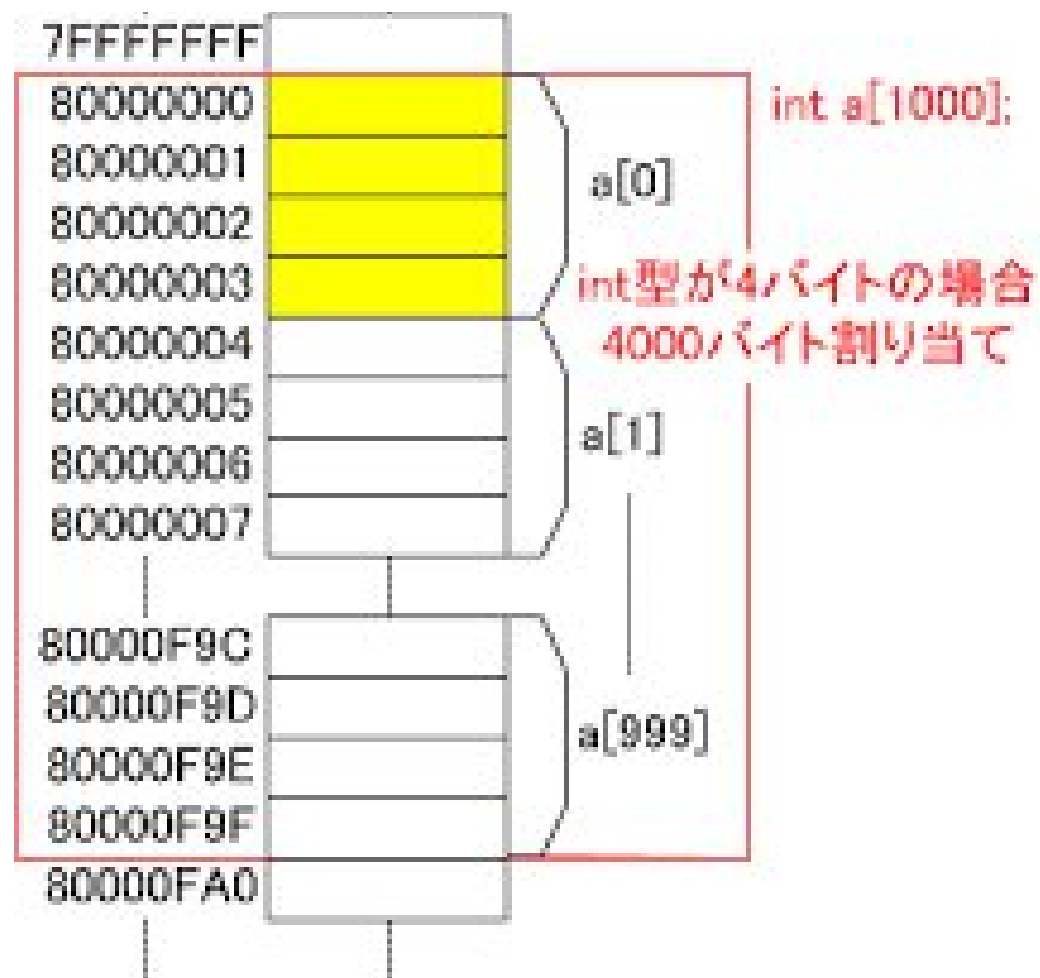


C系列の言語仕様

変数

型	サイズ	説明
<code>bool</code>	1バイト	<code>true</code> / <code>false</code>
<code>byte</code>	1バイト	0～255
<code>sbyte</code>	1バイト	-128～127
<code>char</code>	2バイト	Unicode文字
<code>short</code>	2バイト	整数（小）
<code>ushort</code>	2バイト	符号なし
<code>int</code>	4バイト	整数（基本）
<code>uint</code>	4バイト	符号なし
<code>long</code>	8バイト	大きい整数
<code>ulong</code>	8バイト	符号なし
<code>float</code>	4バイト	単精度小数
<code>double</code>	8バイト	倍精度小数
<code>decimal</code>	16バイト	高精度小数

配列のデータ



変数を配列で定義するときは、メモリーのアドレスを連続するように使用します。

極端な例になりますが、配列 1 0 0 0 を定義するのと、変数 1 0 0 0 個定義するのだと、アドレスとアドレスの間に隙間が無いいためメモリの有効活用ができたリ、アドレスが連番なので、処理が早くなったりします。

3 種類のメモリ領域

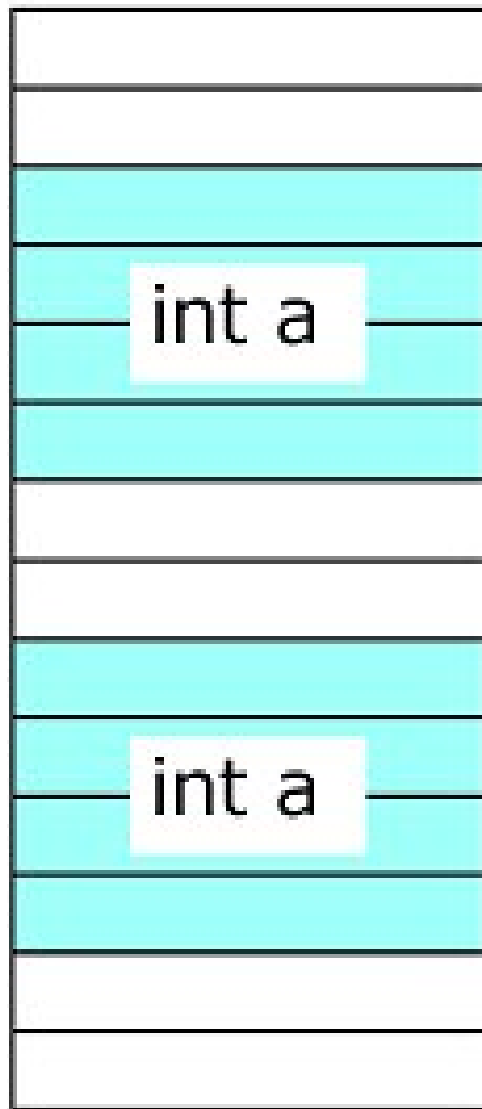
C#で使用されるメモリには3種類あります。

スタック：関数内などで一時的に使用される変数

ヒープ：new やInstanceateなどで中長期に存在する変数

静的領域：static変数など確定でずっと存在する変数

フラグメンテーション



メモリにはフラグメンテーション（断片化）という現象が起こります。

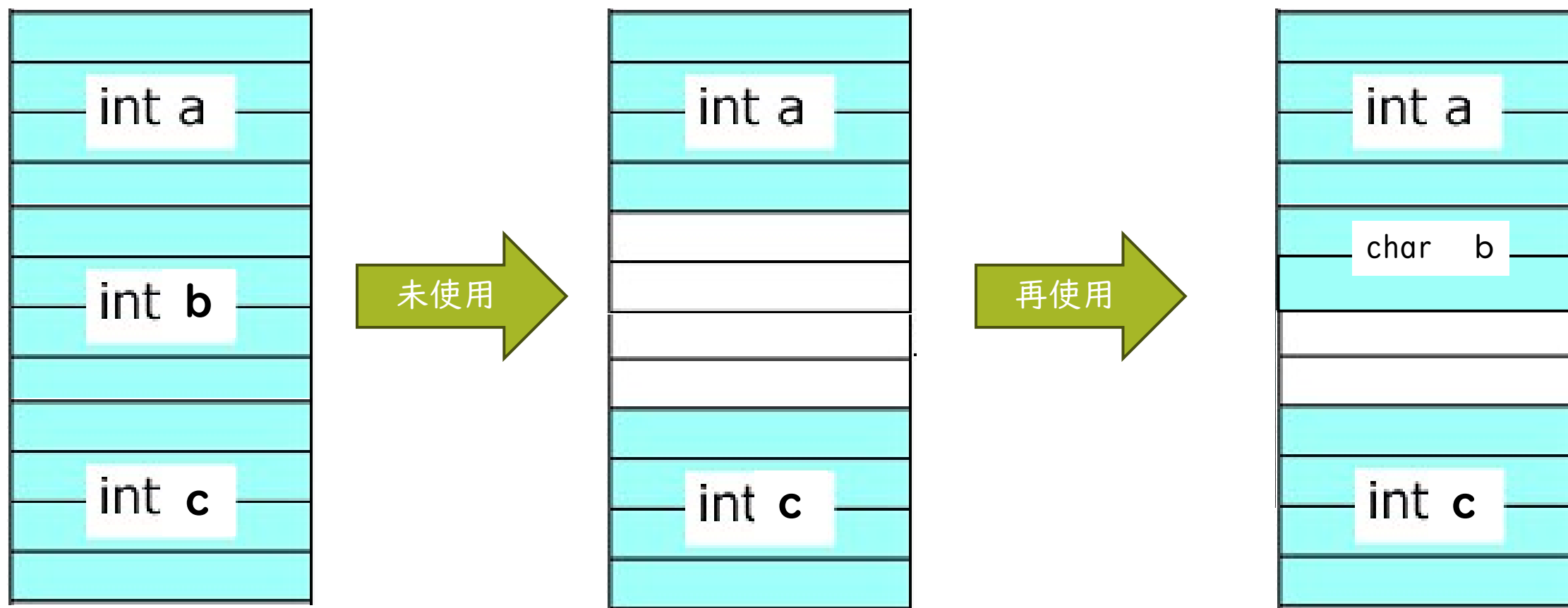
左のメモリを見ると何も入って無いメモリは、上中下に 2 byte ずつの 6 byte あります。

唯、連結した 4 byte が存在しないため、int 型を定義する事が出来ません。

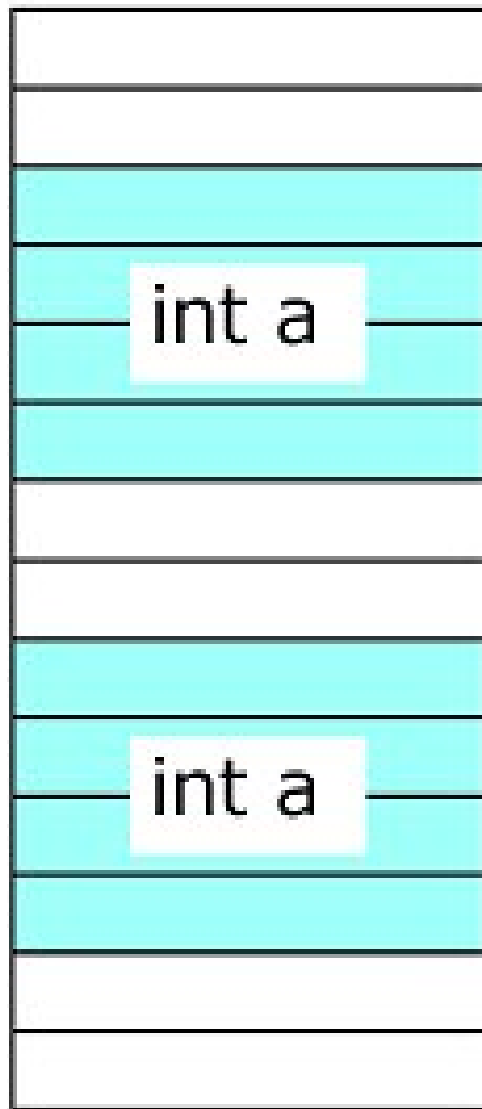
※調査してもメモリは 6 空いてるように表示されるので特に注意が必要

フラグメンテーション

フラグメンテーションは定義されている変数の未使用 $\leftrightarrow$ 使用の切り替わりで発生します。



フラグメンテーション



このフラグメンテーションを低下させるためには、Listなどを作成する際は最初に多めにメモリを用意する

```
List<int> list = new List<int> (1000);
```

繰り返しnew やInstanceateせず、極力使い回す。
などを行うことで抑えることができます。

※完全に0にする事は無理です。

値型と参照型

C#の変数には、住所を扱っている変数と値を扱っている変数があります。

値を扱っている変数は先程バイト数を紹介した基本型の変数と構造体です。これらを値型といいます。

ポインター（アドレス、住所）を扱っている変数は、クラスや配列などnewやInstanceateする必要があるやつです。これらを参照型といいます。

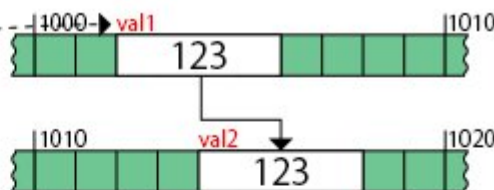
値型と参照型の違い

値型と参照型の大きな違いはコピーを作った際の挙動に現れます。

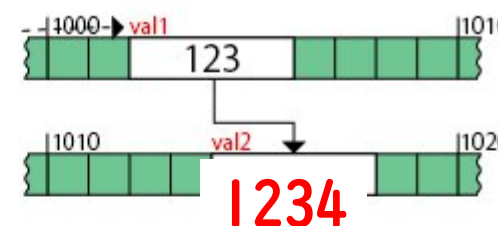
値型

```
int val1 = 123;
```

```
int val2 = val1;
```



val2に変更を加えると



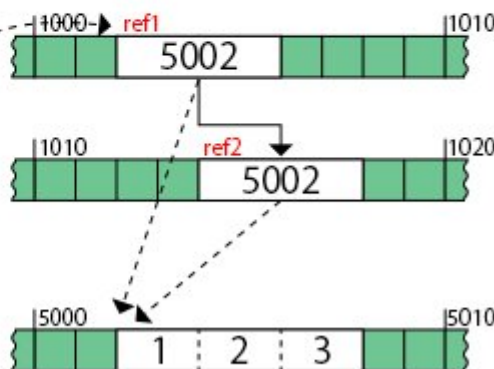
val2が替わる

参照型

```
short[] ref1 =  
new short[] { 1, 2, 3 };
```

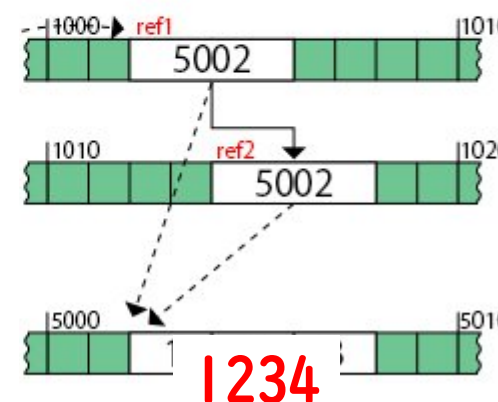
```
short[] ref2 = ref1;
```

変数 ref1 も ref2 も
「5002 番地」にアクセス
するようになる



※アドレスの数値は適当です

ref2に変更を加えると

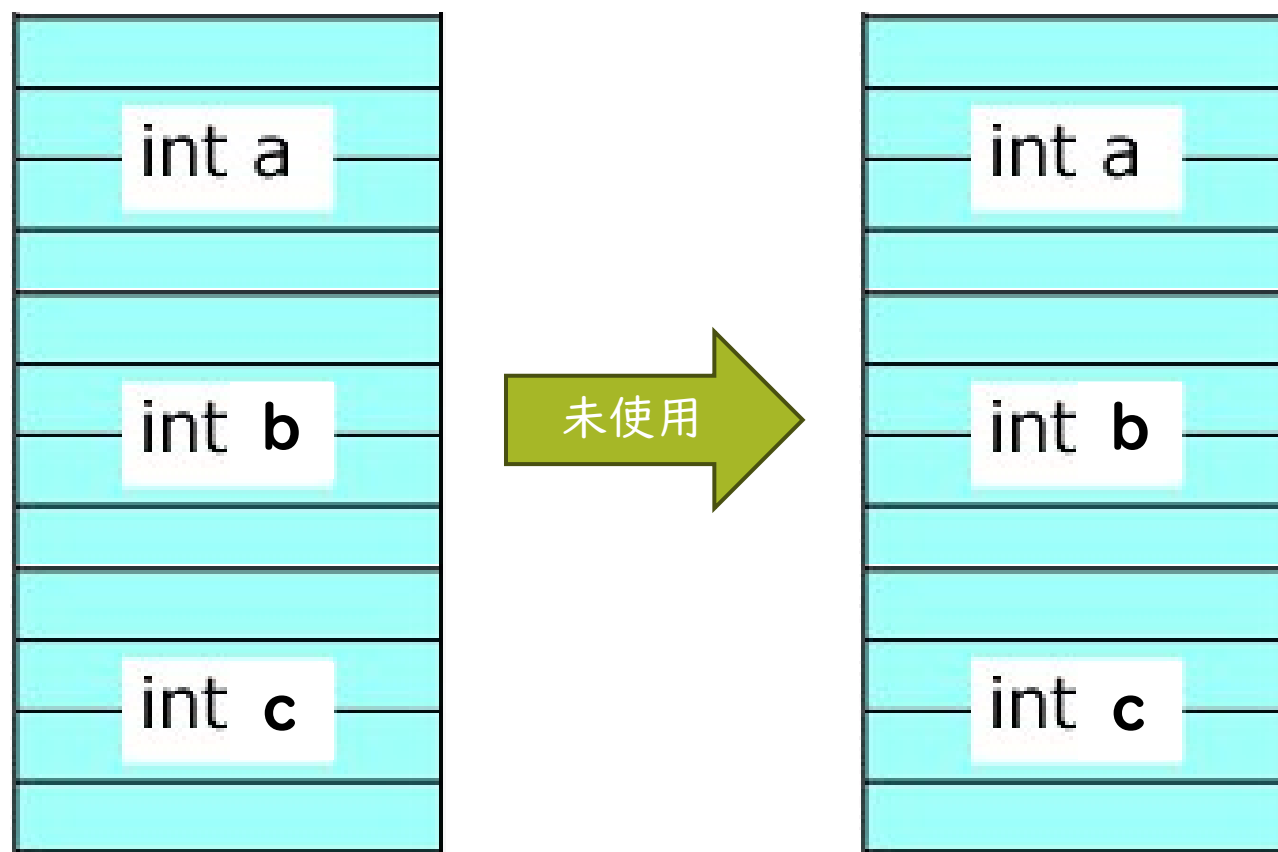


※アドレスの数値は適当です

元データが
替わる

メモリリーク

メモリリークとは、使わなくなったメモリが開放されずに残り続ける事



未使用になったのにメモリの枠を開けない

メモリリーク

メモリリークは主にC++を使っている際にヒープ領域（参照型のある所）で発生します。

先ほど、ヒープ領域は中長期的に使うとありました。

C++では参照型は基本的にいつになったら使われなくなるかが未知数なため、自動的に削除されません。

そのため、自分で使わなくなったタイミングで削除する必要があります。削除せずシーンを切り替えてしまったり、するとこのメモリリークが発生します。

ガベージコレクション

C#ではこのメモリリークを常に監視しています。
newされたクラスを参照するものがなくなったら自動的に破棄してくれるようにシステム側が監視するようになったため、
プログラマーが書き忘れても問題ないようになっています。

がしかし、システム側が常に監視する都合上、
C#そのものの起動がC++に比べて重たくなっています。

この仕組みの事をガベージコレクションといいます。

C++での値型、参照型の話

今まで値型と参照型の話をしてきましたが、これはC#の話です。

C#ではクラスはすべて参照型でintはすべて値型のように決まって居ますが、C++ではクラスの値型やintの参照型のようにどちらでも定義できます。

クラスを値型で定義するとヒープではなくスタックに保存されるため、

メモリリークし難い代わりに、コピー的な挙動をするので、扱いにくいなどそれぞれの定義方法にメリット・デメリットがあります。唯、体験する機会が無い上にゲキムズなので割愛します。